РУКОВОДСТВО / ЛОКАЛЬНЫЙ AI

БРОДЯГА: установка и работа

Пошаговый путь от чистой системы до локального чата, Qwen 3 / Qwen 3 VL и Kimi Code — без облачной обязаловки и без догадок.

ГОД	ВЕРСИЯ	ВЛАДЕЛЕЦ
_____	1.0	media314

ДАТА ВЫПУСКА	СТАТУС	РЕПОЗИТОРИЙ
2026-08-23	PUBLIC	media314/BRODYAGA

НАВИГАЦИЯ

Как пользоваться руководством

Этот PDF написан для человека, который умеет открыть терминал, но ещё не обязан понимать Docker, CUDA, VLM или API.

ПРОЕКТ	ГОД	ОТВЕТСТВЕННЫЙ
_____	_____	_____

1 · ВЫБЕРИТЕ РЕЖИМ
CPU, NVIDIA GPU или native Ollama.

2 · ПОДГОТОВЬТЕ ХОСТ
Git, Docker, драйвер и toolkit по ситуации.

3 · ЗАПУСТИТЕ СТЕК
Bootstrap создаст конфиг и скачает модели.

4 · ВОЙДИТЕ В ЧАТ
Первая учётная запись становится администратором.

5 · ПОДКЛЮЧИТЕ KIMI
Агент использует тот же Ollama API.

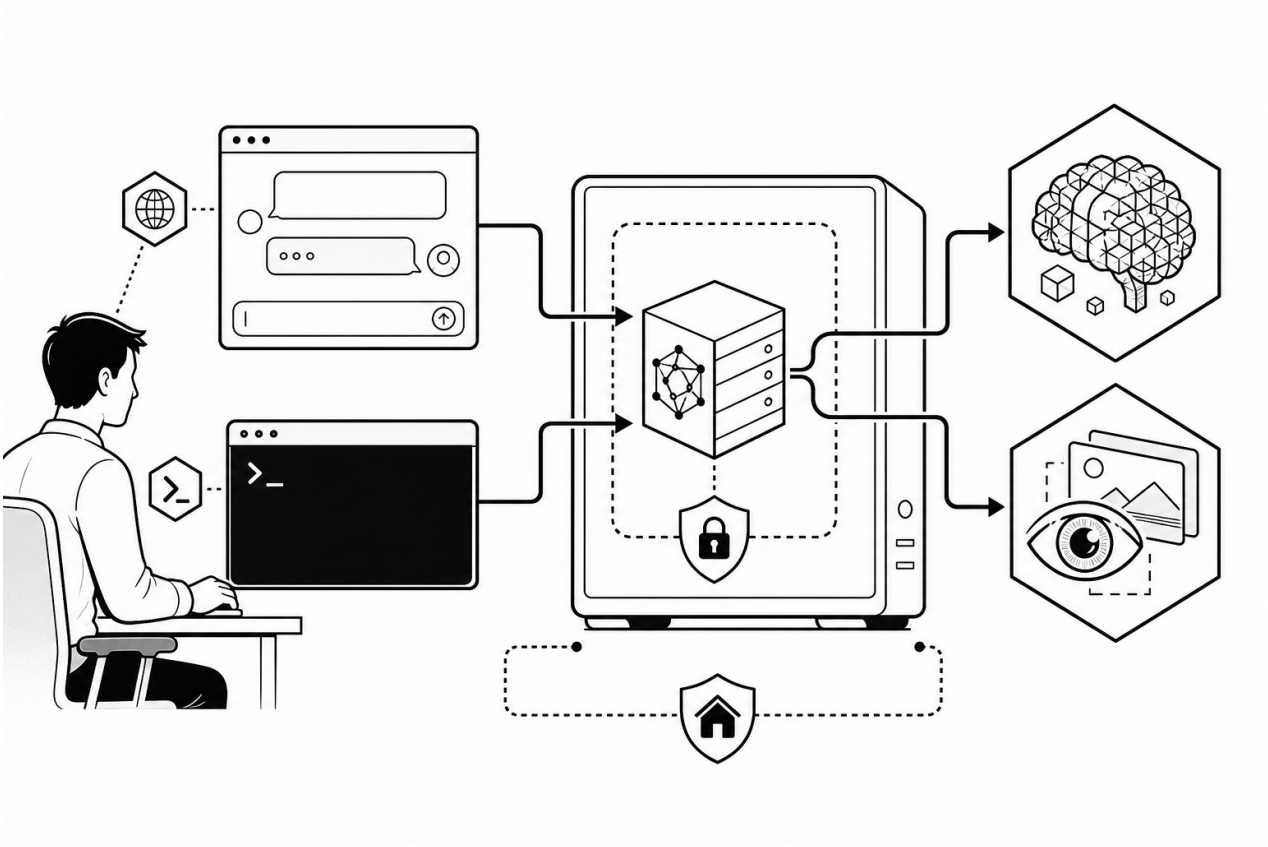
6 · ПРОВЕРЬТЕ ПАМЯТЬ
Для 3060 цель — одна модель и 100% GPU.

> **«Копируйте команды по одному блоку и проверяйте результат перед следующим шагом.»**
>
> ГЛАВНОЕ ПРАВИЛО ДЛЯ НОВИЧКА

АРХИТЕКТУРА

Что именно вы запускаете

БРОДЯГА объединяет два интерфейса и один локальный inference-сервер. Все связи по умолчанию остаются на вашем компьютере.



OPEN WEBUI
Чат в браузере на localhost:3000.

KIMI CODE
Coding agent в терминале и папке проекта.

OLLAMA
Локальный API на localhost:11434.

QWEN 3
Текст, рассуждение, инструменты и код.

QWEN 3 VL
Изображения, скриншоты, документы и OCR.

LOOPBACK
Порты 127.0.0.1 не открыты наружу.

> **«Один сервер, два интерфейса, одна библиотека моделей.»**

АРХИТЕКТУРА

Где живут программы и данные

Понимание четырёх мест снимает половину типичных вопросов: хост, контейнеры, volumes и рабочая папка репозитория.

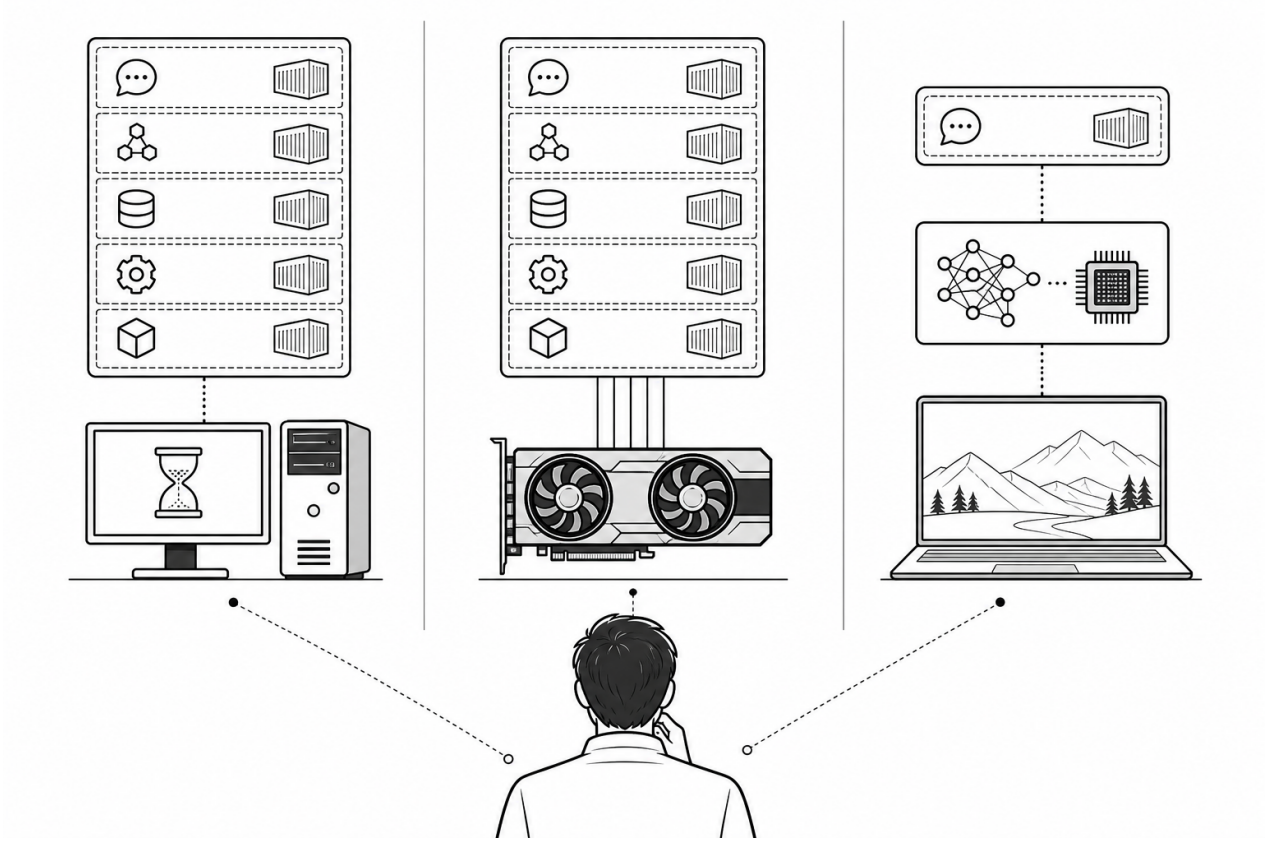
МЕСТО	ЧТО ТАМ	ЧТО ВАЖНО
Хост	NVIDIA driver, Docker, Git; в native-режиме ещё Ollama	Драйвер связывает ОС с GPU. Полный CUDA Toolkit стеку не нужен.
Контейнер Ollama	Модели и API	В GPU-режиме получает видеокарту через NVIDIA Container Toolkit.
Контейнер Open WebUI	Интерфейс, аккаунты, история	Обращается к Ollama по внутреннему имени ollama:11434.
Docker volumes	Веса моделей, настройки, чаты	make down их сохраняет; down -v удаляет данные.
.env	Модели, порты, контекст, параллельность	Не коммитить. Bootstrap создаёт файл из .env.example.
.runtime/kimi	Изолированный конфиг Kimi	Не меняет глобальный ~/.kimi-code/config.toml.

> **«Контейнер — это процесс с настройками. Volume — это данные, которые переживают перезапуск.»**

ВЫБОР РЕЖИМА

CPU, NVIDIA GPU или native

Выберите один путь до запуска. Режимы независимы: один не является обязательной ступенью другого.



РЕЖИМ	КОМУ ПОДХОДИТ	КОМАНДА
CPU	Любой компьютер; медленно, но предсказуемо	./scripts/bootstrap.sh --cpu
GPU	Linux / WSL2 + NVIDIA	./scripts/bootstrap.sh --gpu
Native	macOS / Windows или Ollama уже на хосте	./scripts/bootstrap.sh --native

ТРЕБОВАНИЯ

Сколько нужно памяти

Цифры ниже — практический старт для одной активной модели и одного запроса. Контекст и параллельность расходуют дополнительную память.

ПРОФИЛЬ	МОДЕЛИ Q4	RAM	VRAM	ДИСК
Лёгкий	Qwen 3 4B + VL 4B	16 ГБ	6–8 ГБ	15+ ГБ
Рекомендуемый	Qwen 3 8B + VL 8B	32 ГБ	10–12 ГБ	25+ ГБ
Мощный	Qwen 3 30B + VL 30B	64 ГБ	24+ ГБ	55+ ГБ

RTX 3060 12 ГБ

КОМПОНЕНТ	МИНИМУМ	РЕКОМЕНДУЕТСЯ	ПОЧЕМУ
GPU	RTX 3060 12 ГБ	10+ ГБ свободной VRAM	Монитор и браузер тоже занимают VRAM
Driver	550+	Актуальный стабильный	Текущее требование Ollama для CC 8.6
RAM	16 ГБ	32 ГБ	Docker, UI, cache и запас без swap
CPU	4 ядра	6+ ядер / 12 потоков	Загрузка и вспомогательные задачи
SSD	25 ГБ	40 ГБ	Образы, две 8B-модели, обновления
ОС	Linux x86_64 / WSL2	Ubuntu 24.04 LTS	Простой путь Docker + NVIDIA

ПОДГОТОВКА

Проверка перед установкой

Не начинайте с Docker Compose. Сначала убедитесь, что ОС видит железо и что выбранный режим реалистичен.

ОС
uname -m и cat /etc/os-release

GPU
lspci | grep -i nvidia

ДРАЙВЕР
nvidia-smi

DOCKER
docker --version

COMPOSE V2
docker compose version

ДИСК
df -h

```
uname -m
cat /etc/os-release
lspci | grep -i nvidia
nvidia-smi
docker --version
docker compose version
docker info
```

- Если nvidia-smi не работает, не переходите к Container Toolkit: сначала исправьте драйвер.
- На macOS CUDA не существует — используйте native Ollama и Metal.
- Для CPU-режима NVIDIA driver и Container Toolkit не нужны.

> **«Каждый следующий слой проверяется только после предыдущего.»**

NVIDIA

Драйвер на Ubuntu

Пример ниже рассчитан на Ubuntu 24.04. Пакеты и поддержка меняются: перед установкой сверяйтесь с официальной инструкцией NVIDIA.

```
DISTRO=ubuntu2404
sudo apt-get update
sudo apt-get install -y linux-headers-$(uname -r) wget
wget https://developer.download.nvidia.com/compute/cuda/repos/\
${DISTRO}/x86_64/cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt-get update
sudo apt-get install -y nvidia-open
sudo reboot
```

ПОСЛЕ ПЕРЕЗАГРУЗКИ

```
nvidia-smi
cat /proc/driver/nvidia/version
```

- Для RTX 3060 ориентир — NVIDIA driver 550 или новее.
- Если open kernel modules не поддерживаются, используйте рекомендованный NVIDIA пакет cuda-drivers.
- Secure Boot может потребовать регистрацию ключа MOK; сообщение установщика нельзя пропускать.
- Строка CUDA в nvidia-smi показывает максимум runtime драйвера, а не установленный CUDA Toolkit.

> **«Для запуска Ollama в Docker полный CUDA Toolkit на хосте не нужен.»**

NVIDIA

NVIDIA Container Toolkit

Драйвер уже должен работать. Toolkit даёт Docker-контейнеру доступ к видеокарте.

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg2
KEYRING=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
LIST_URL=https://nvidia.github.io/libnvidia-container/stable/deb/\
nvidia-container-toolkit.list
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
  | sudo gpg --dearmor -o "$KEYRING"
curl -sL "$LIST_URL" \
  | sed "s#deb https://#deb [signed-by=$KEYRING] https://#g" \
  | sudo tee /etc/apt/sources.list.d/nvidia-ctk.list
sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

КОНТРОЛЬНЫЙ ТЕСТ

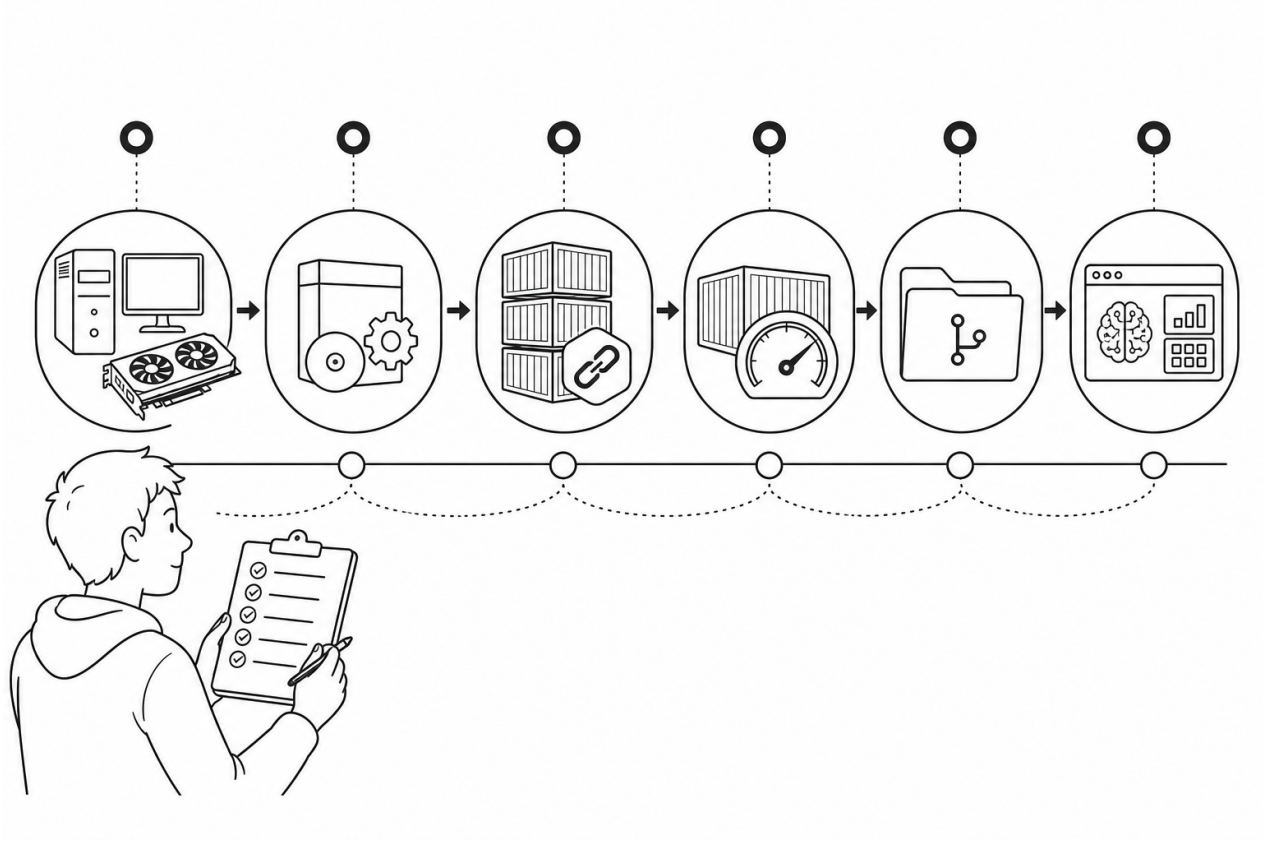
```
docker run --rm --gpus all ubuntu nvidia-smi
```

- Если тестовый контейнер не видит GPU, Ollama тоже не увидит его.
- В WSL2 драйвер ставится в Windows; Linux display driver поверх него внутри WSL не устанавливайте.
- После настройки runtime Docker daemon обязательно перезапускается.

УСТАНОВКА

Весь путь одной схемой

Шесть шагов идут строго слева направо. Подписи под схемой — ваша контрольная карта.



1 · ЖЕЛЕЗО
ОС видит RTX 3060.

2 · ДРАЙВЕР
nvidia-smi без ошибки.

3 · TOOLKIT
Docker получил runtime NVIDIA.

4 · GPU TEST
Контейнер видит карту.

5 · РЕПОЗИТОРИЙ
Клон и локальный .env.

6 · ЗАПУСК
UI, модели и health checks.

УСТАНОВКА

Клонирование и первый запуск

Работайте из обычной пользовательской папки. Не запускайте bootstrap через sudo: Docker должен быть настроен для вашего пользователя.

```
git clone https://github.com/media314/BRODYAGA.git
cd BRODYAGA
./scripts/doctor.sh
```

LINUX / WSL2 + NVIDIA

```
./scripts/bootstrap.sh --gpu
```

СЛАБАЯ МАШИНА

```
./scripts/bootstrap.sh --gpu --light
# или: ./scripts/bootstrap.sh --cpu --light
```

MACOS / WINDOWS С NATIVE OLLAMA

```
ollama serve
./scripts/bootstrap.sh --native
```

- Bootstrap создаёт .env из .env.example, запускает сервисы и скачивает две модели.
- Стандартные 8B-веса суммарно занимают около 11.3 ГБ на диске; загрузка может быть долгой.
- Чтобы запустить сервисы без моделей, добавьте --skip-models и позже выполните make pull.

ПЕРВЫЙ ВХОД

Откройте чат и зафиксируйте администратора

После окончания bootstrap интерфейс доступен по адресу http://localhost:3000.

ОТКРОЙТЕ
http://localhost:3000

СОЗДАЙТЕ АККАУНТ
Первая учётная запись станет admin.

ВЫБЕРИТЕ МОДЕЛЬ
qwen3:8b для обычного диалога.

СДЕЛАЙТЕ ЗАПРОС
Первый ответ загрузит модель в память.

ПРОВЕРЬТЕ
make doctor и make memory.

ЗАКРОЙТЕ SIGNUP
ENABLE_SIGNUP=false после регистрации.

```
make doctor
make memory
./scripts/smoke-test.sh
```

- localhost доступен только на этом компьютере; это безопасное значение по умолчанию.
- Не меняйте bind на 0.0.0.0 без TLS, reverse proxy и продуманной аутентификации.
- Чаты и настройки сохраняются в volume open-webui-data.

> **«Первый рабочий результат — не красивый экран, а успешный health check и ответ модели.»**

OPEN WEBUI

Как работать в чате каждый день

Начните с простой задачи. Выбор модели зависит от входных данных, а не от названия вкладки.

ЗАДАЧА	МОДЕЛЬ	КАК ДЕЙСТВОВАТЬ
Текст, план, письмо, код	qwen3:8b	Опишите цель, контекст, ограничения и формат ответа.
Скриншот, фото, PDF-страница	qwen3-vl:8b	Прикрепите файл и укажите, что именно найти или сравнить.
Быстрый ответ на слабой машине	4B	Переключитесь на light-профиль или измените .env.
Длинный анализ	8B / больше	Следите за контекстом: он расходует KV-cache.

ШАБЛОН ХОРОШЕГО ЗАПРОСА

```
Цель: что должно получиться.
Контекст: какие данные и для кого.
Ограничения: что нельзя менять или выдумывать.
Формат: таблица, чек-лист, код, краткий ответ.
Проверка: по каким признакам ответ верный.
```

- Для VLM укажите область изображения: весь экран, таблица, ошибка или конкретный элемент.
- Не отправляйте модели секреты только потому, что стек локальный: чаты всё равно сохраняются.
- В сборке нет отдельного moderation-сервиса; поведение зависит от весов модели и системного промпта.

МОДЕЛИ

Выберите размер без самообмана

Размер файла — только веса. Сверху нужны KV-cache, runtime, vision encoder и буферы.

МОДЕЛЬ	ВЕС Q4	RTX 3060 12 ГБ	ПРИМЕНЕНИЕ
qwen3:4b	2.5 ГБ	Большой запас	Быстрый текст, короткие coding-задачи
qwen3-vl:4b	3.3 ГБ	Большой запас	Лёгкий vision и OCR
qwen3:8b	5.2 ГБ	Да, одна модель	Основной чат и Kimi
qwen3-vl:8b	6.1 ГБ	Да, одна модель	Основной визуальный разбор
qwen3:14b	9.3 ГБ	Пограничено	Только короткий контекст; offload возможен
qwen3:30b	19 ГБ	Нет	Веса уже больше VRAM

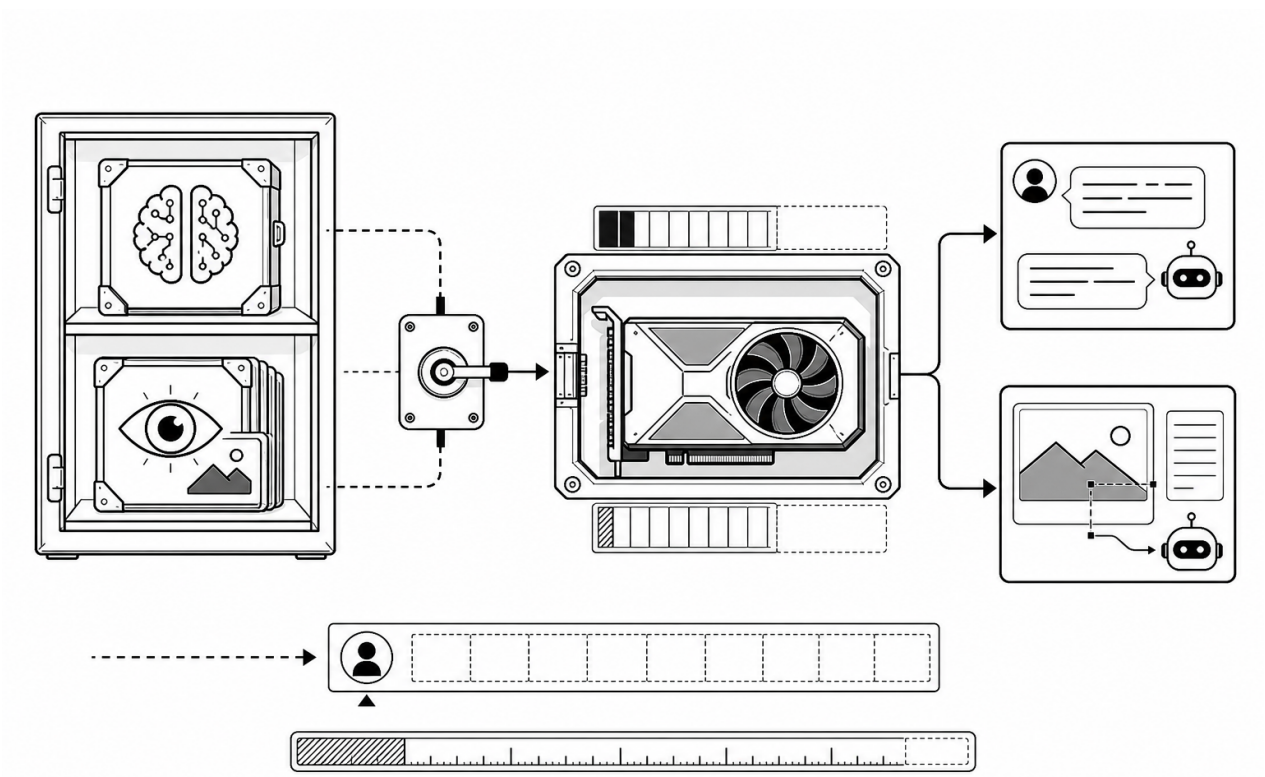
- Для 3060 выбирайте 8B как потолок комфортного старта и держите одну модель.
- VLM загружается вместо LLM; небольшая пауза при переключении — ожидаемое поведение.
- Заявленное окно 256K не означает, что такой контекст помещается в 12 ГБ VRAM.
- Если важнее скорость и стабильность, 4B часто полезнее пограничной 14B.

> **«Контекст — это бюджет памяти, а не бесплатный максимум.»**

МОДЕЛИ

Одна активная модель на RTX 3060

Обе 8B-модели хранятся на диске, но в VRAM загружается только та, к которой пришёл текущий запрос.



БИБЛИОТЕКА
Веса LLM и VLM лежат на диске.

ПЕРЕКЛЮЧАТЕЛЬ
Ollama выгружает предыдущую модель.

VRAM
Одна модель + её KV-cache.

ТЕКСТ
Qwen 3 отвечает в чате и Kimi.

ЗРЕНИЕ
Qwen 3 VL получает изображение.

ОЧЕРЕДЬ
Один запрос не умножает память.

> **«Одна активная модель — это контроль бюджета, а не поломка.»**

НАСТРОЙКА

Параметры .env простыми словами

Для RTX 3060 готовый профиль уже содержит безопасные значения. Если настраиваете вручную, меняйте один параметр за раз.

```
QWEN_MODEL=qwen3:8b
VLM_MODEL=qwen3-vl:8b
OLLAMA_CONTEXT_LENGTH=16384
OLLAMA_KEEP_ALIVE=0
OLLAMA_MAX_LOADED_MODELS=1
OLLAMA_NUM_PARALLEL=1
OLLAMA_FLASH_ATTENTION=1
OLLAMA_KV_CACHE_TYPE=q8_0
```

ПАРАМЕТР	ЧТО ДЕЛАЕТ	ДЛЯ 3060
CONTEXT_LENGTH	Максимум токенов и размер KV-cache	16K; при offload снизить до 8K
KEEP_ALIVE	Сколько модель остаётся загруженной	0 — выгрузить после ответа
MAX_LOADED_MODELS	Число моделей одновременно	1
NUM_PARALLEL	Параллельные запросы на модель	1
FLASH_ATTENTION	Снижает рост памяти на длинном контексте	1
KV_CACHE_TYPE	Точность и размер KV-cache	q8_0 ≈ половина f16

QWEN 3 VL

Как разбирать изображения

VLM не читает ваши мысли. Передайте изображение и задайте наблюдаемую задачу.

1 · ПРИКРЕПИТЕ
Скриншот, фото или страницу документа.

2 · УКАЖИТЕ ОБЛАСТЬ
Например, правая панель или строка ошибки.

3 · СФОРМУЛИРУЙТЕ ЦЕЛЬ
Распознать, сравнить, извлечь или объяснить.

4 · ЗАДАЙТЕ ФОРМАТ
Таблица полей, список дефектов, краткое резюме.

5 · ПОПРОСИТЕ ДОКАЗАТЕЛЬСТВО
Сослаться на видимые элементы.

6 · ПРОВЕРЬТЕ
OCR и числа сверяются человеком.

```
Разбери приложенный скриншот.
Цель: найти причину ошибки авторизации.
Сначала перечисли только видимые факты.
Потом дай 3 гипотезы от наиболее вероятной.
Не выдумывай текст, которого не видно.
```

- Для длинных документов загружайте релевантные страницы, а не весь архив без задачи.
- При переключении с qwen3:8b на qwen3-vl:8b Ollama может заново загрузить веса.
- VLM-output не является источником истины для критических чисел, дат и юридических фактов.

> **«Сначала видимые факты, затем интерпретация.»**

KIMI CODE

Установка coding agent

Kimi Code — отдельный терминальный клиент. БРОДЯГА подключает его к локальному Ollama через OpenAI-compatible API.

MACOS / LINUX

```
./scripts/install-kimi.sh
kimi --version
```

ОФИЦИАЛЬНЫЙ ЭКВИВАЛЕНТ

```
curl -fsSL https://code.kimi.com/kimi-code/install.sh | bash
```

WINDOWS POWERSHELL

```
irm https://code.kimi.com/kimi-code/install.ps1 | iex
```

- На Windows заранее установите Git for Windows: Kimi использует Git Bash.
- Локальный скрипт сначала скачивает официальный installer во временный файл и только потом запускает его.
- После установки откройте новый терминал, если команда kimi ещё не появилась в PATH.
- Официальная облачная модель подключается отдельно: запустите kimi и выполните /login.

> **«Установка Kimi не запускает модель: модель уже обслуживает Ollama.»**

KIMI CODE

Подключение локального Qwen

Самый безопасный путь — обёртка репозитория. Она создаёт изолированный конфиг и включает ручные подтверждения.

```
./scripts/kimi-local.sh .
# профиль RTX 3060:
make kimi-3060
# VLM для скриншотов:
./scripts/kimi-local.sh --model qwen3-vl:8b .
```

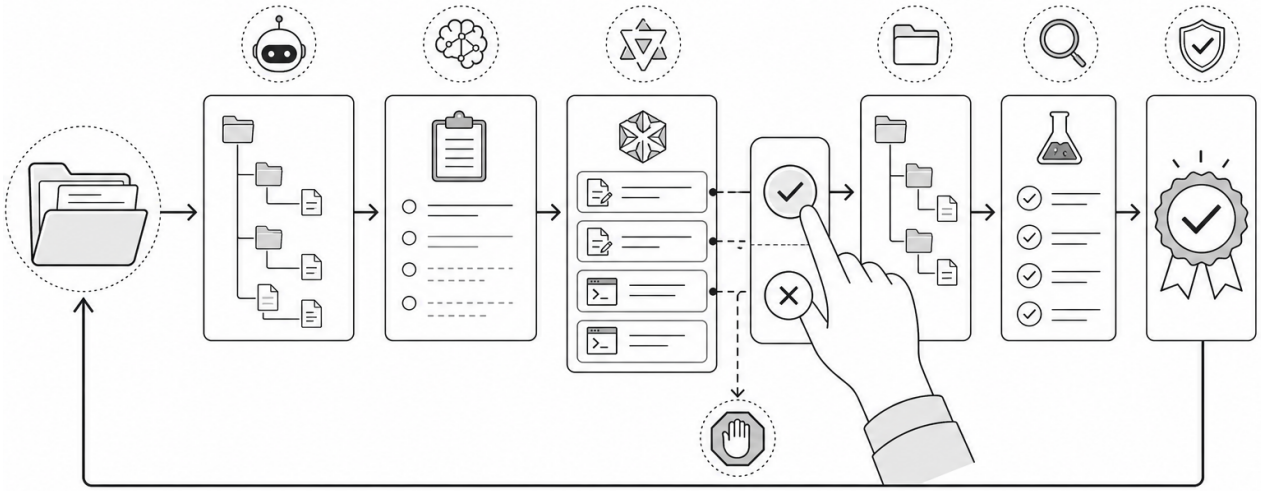
ПОЛЕ	ЗНАЧЕНИЕ	СМЫСЛ
type	openai	Kimi использует Chat Completions protocol
base_url	http://127.0.0.1:11434/v1	OpenAI-compatible endpoint Ollama
api_key	ollama	Непустая заглушка; локальный Ollama не проверяет
permission	manual	Команды и правки подтверждает человек
context	16384	Совпадает с безопасным профилем 3060

- Конфиг создаётся в .runtime/kimi и не трогает глобальный файл пользователя.
- Для удалённого Ollama передайте --base-url, но открывайте API только в доверенной сети.
- Coding agent сильнее зависит от tool calling, чем чат; маленькая модель может ошибаться чаще.

KIMI CODE

Как агент меняет проект

Рабочий цикл остаётся под контролем человека: изучение, план, предложение, approval, изменение и проверка.



ПАПКА
Вы выбираете точный проект.

АНАЛИЗ
Агент читает структуру и контекст.

ПЛАН
Действия формулируются до правок.

APPROVAL
Команда проходит ручной gate.

ИЗМЕНЕНИЕ
Разрешённые файлы обновляются.

ПРОВЕРКА
Тесты и diff закрывают цикл.

> **«Kimi предлагает; человек решает.»**
>
> РЕЖИМ MANUAL

KIMI CODE

Безопасная ежедневная работа

Не начинайте просьбой «исправь всё». Дайте агенту узкую цель и критерий готовности.

1 · ОТКРОЙТЕ ПРОЕКТ
Запускать Kimi из нужной папки.

2 · ДАЙТЕ ЗАДАЧУ
Один результат, ясные границы.

3 · ПОПРОСИТЕ ПЛАН
Файлы, риски, проверки.

4 · ЧИТАЙТЕ DIFF
Не подтверждать незнакомые команды.

5 · ЗАПУСТИТЕ ТЕСТЫ
Команды реального репозитория.

6 · ЗАФИКСИРУЙТЕ ИТОГ
Что изменено и что осталось.

```
Изучи репозиторий и сначала дай план.
Цель: добавить команду диагностики без изменения данных.
Не трогай файлы вне scripts/ и docs/.
Перед командами объясняй риск.
Готово, когда make test проходит.
```

- Не включайте yolo/auto для незнакомого проекта или новой локальной модели.
- Секреты, .env и ключи не должны попадать в prompt, diff или commit.
- Если агент предлагает удаление, остановитесь и проверьте точный target и возможность восстановления.
- Для сложной coding-задачи официальный Kimi или большая модель может быть надёжнее локальной 8B.

ПРАКТИКА

Три готовых сценария

Эти шаблоны можно копировать в Open WebUI или Kimi и подставлять свои данные.

ЧАТ

```
Объясни тему для новичка.
Сначала дай аналогию, затем 5 шагов и чек-лист.
Новые термины объясняй при первом употреблении.
```

VLM

```
Разбери изображение. Отдели факты от предположений.
Верни таблицу: область / наблюдение / риск / проверка.
Нечитаемые фрагменты пометь как неопределённые.
```

KIMI

```
Сначала прочитай AGENTS.md и релевантные файлы.
Ничего не меняй до плана. Сохрани пользовательские правки.
После изменений запусти проектные проверки и покажи итог.
```

- Если ответ слабый, уточняйте входные данные, а не только просите «подумай лучше».
- Большой контекст не исправляет плохую постановку задачи.
- Локальность не отменяет ответственность владельца за законное и безопасное использование.

ПАМЯТЬ

RTX 3060: режим без offload

Цель — веса и KV-cache одной модели находятся в VRAM, а системная RAM не используется для вычислительного offload.

```
make rtx3060
# отправьте один запрос модели
make memory
make kimi-3060
```

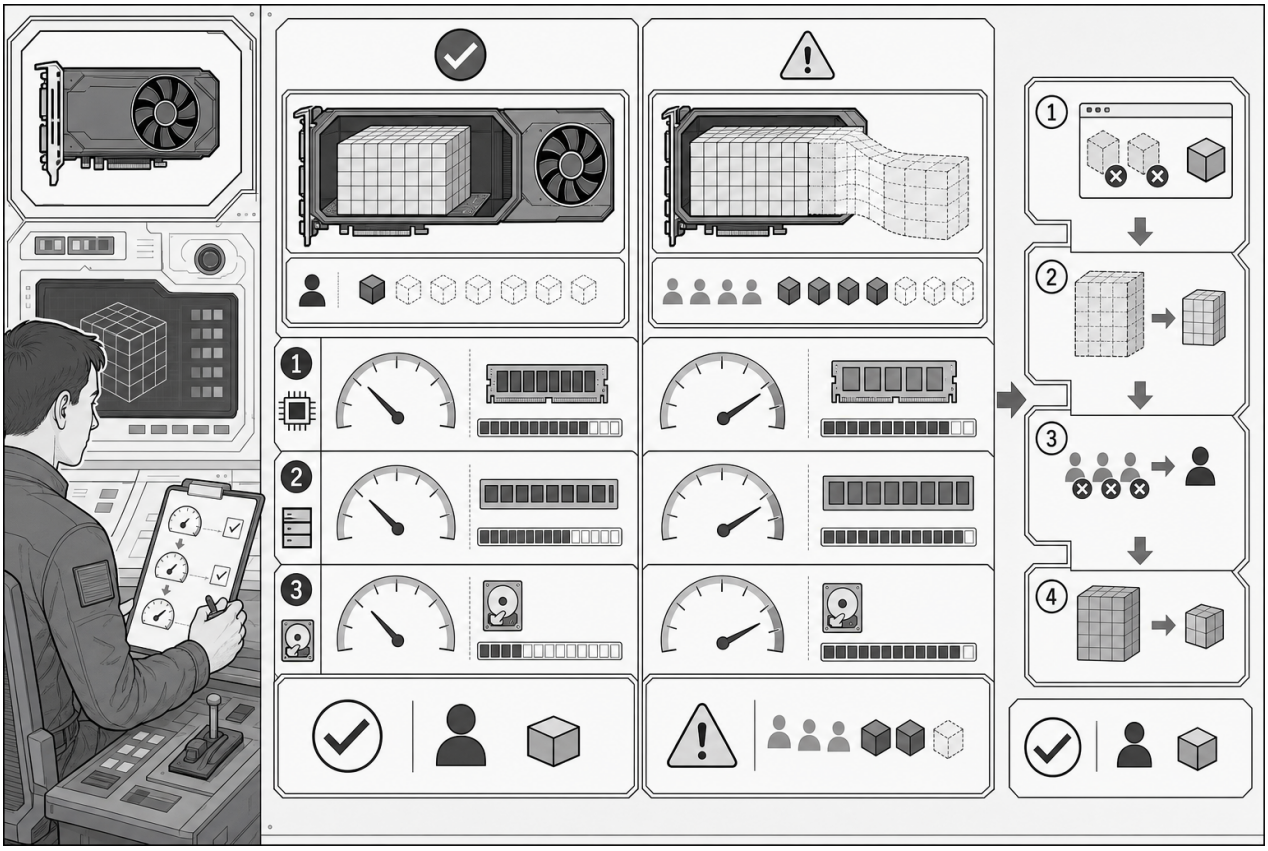
ПРОВЕРКА	ЗДОРОВОЕ ЗНАЧЕНИЕ	ЕСЛИ НЕ ТАК
ollama ps	PROCESSOR = 100% GPU	Уменьшить контекст или модель
Loaded models	Одна	MAX_LOADED_MODELS=1
Context	16384	Снизить до 8192
Parallel	1	NUM_PARALLEL=1
docker stats	Ollama <12 ГБ; UI <4 ГБ	Проверить memory overlay
free -h	available стабильно	Закрыть процессы; проверить swap
nvidia-smi	Нет лишних GPU-приложений	Закрыть игры и GPU-ускоренный браузер

- 100% GPU не означает 0 байт RAM: Docker, UI, драйвер, буферы и файловый cache используют память.
- q8_0 KV-cache занимает примерно половину f16 с очень небольшой потерей точности.
- KEEP_ALIVE=0 освобождает модель после ответа, но следующий первый токен будет медленнее.

ПАМЯТЬ

Что проверять, если «жрёт оперативу»

Смотрите не на одну цифру, а на размещение модели, лимиты контейнеров, available RAM и swap.



VRAM
Модель целиком на GPU.

HOST RAM
Оценивайте available, а не used.

SWAP
Не должен расти во время ответа.

ЗАКРЫТЬ ЛИШНЕЕ
Освободить GPU-процессы.

УМЕНЬШИТЬ
Контекст 16K → 8K.

УПРОСТИТЬ
8B → 4B, один запрос.

> **«100% GPU — цель; нулевое потребление RAM — миф.»**

ДИАГНОСТИКА

Если что-то не работает

Идите сверху вниз. Не меняйте несколько параметров одновременно — иначе причина потеряется.

СИМПТОМ	ПРОВЕРКА	СЛЕДУЮЩЕЕ ДЕЙСТВИЕ
UI не открывается	docker compose ps; logs open-webui; curl /health	Проверить порт; WEBUI_PORT=3001
UI не видит модели	curl :11434/api/tags; ollama list	make pull; внутри Docker URL = ollama:11434
GPU не используется	nvidia-smi; Docker runtime; test container	Переустановить/настроить Container Toolkit
Очень медленно / OOM	ollama ps; make memory	Контекст 8K/16K; одна модель; 4B
Kimi без provider	Проверить запуск через wrapper	./scripts/kimi-local.sh .
Kimi ломает tool calls	./scripts/smoke-test.sh	Снизить контекст; выбрать сильнее модель

```
./scripts/doctor.sh
make logs
make memory
./scripts/smoke-test.sh
make test
```

- make down останавливает сервисы и сохраняет модели, аккаунты и чаты.
- Не используйте docker compose down -v без осознанного решения удалить данные.
- После suspend/resume Linux при потере GPU может помочь перезагрузка модуля nvidia_uvm или системы.

ФИНАЛ

Рабочая памятка

Пять команд закрывают обычный жизненный цикл. Сохраните эту страницу рядом с компьютером.

КОГДА	КОМАНДА	ЧТО ПОЛУЧИТЕ
Первый запуск 3060	make rtx3060	Стек, лимиты и две 8B-модели
Проверка	make doctor	Состояние хоста, GPU, API и Kimi
Контроль памяти	make memory	VRAM, offload, container RAM
Логи	make logs	Последние события Ollama и UI
Остановка	make down	Сервисы остановлены, данные сохранены

ПРОВЕРЕННЫЕ ИСТОЧНИКИ · 2026-08-23

Open WebUI Quick Start
https://docs.openwebui.com/getting-started/quick-start/

Ollama GPU / FAQ / OpenAI compatibility
https://docs.ollama.com/gpu

NVIDIA Container Toolkit
https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html

Kimi Code Getting Started
https://moonshotai.github.io/kimi-code/en/guides/getting-started

Kimi providers and models
https://moonshotai.github.io/kimi-code/en/configuration/providers

Qwen 3 VL in Ollama
https://ollama.com/library/qwen3-vl

> **«Курсантье всегда придёт на помощь!»**
>
> ПОМЕТКА НА ПОЛЯХ